Annex XIII — Hybrid AI in Inhomogeneous Corporate Environments

Optirando™ Canon — Annex XIII, Version 1.0 Date: 25 July 2026 Status: Defensive publication. Normative within the Optirando™ Canon.

Key words *MUST, MUST NOT, SHOULD, SHOULD NOT,* and *MAY* are to be interpreted as constraints on conforming implementations of the architecture described in this Canon.

XIII.1 Scope

This annex specifies how heterogeneous AI capabilities are composed and orchestrated inside a single governed workflow, and how the model adaptations that perform that composition are admitted, routed, swapped, and adapted over time.

It specifies four things:

1. the **Composition Adapter** — an admitted local model adaptation whose function is to construct evidence, not to answer questions;
2. the **Evidence Object** and its **Recipient Projections** — complete within the local trust domain, minimized toward every recipient;
3. the **adapter admission, routing, and adaptation lifecycle**, bound to PoAC™ — including adaptation itself as an authorized, method-declared, lineage-recorded execution whose product carries a residency constraint;
4. **capability resolution** — declaration of every participating provider including spatial models and generation pipelines, eligibility as a hard gate preceding any quality-based selection, policy-bounded routing proposals presented comparatively and never executable without admission, and the non-authorizing **Capability Request** loop.

XIII.1.1 What this annex does not re-specify

This annex is deliberately narrow. The following are specified elsewhere in the Canon and are referenced, not restated:

Subject	Location
WR Handshake architecture, profiles, capability declaration, Merkle manifest, publisher attestation P0/P1/P2	Annex VII
Initiation layer, consent classes C0–C4, WR TED, finalizer lifecycle, verification tiers L/C/X	Annex IX
Internal Handshake, External Service Admission, WR Guard™, WR Expert™, the ten egress measures, Boundary Event Records	Annex X
Hybrid preparation/presentation split, Session Composition Pool, deterministic composition, transparent rendering pipeline	Annex VIII
PoAC™ capability acquisition, gap typology, evidence requirement, escalation ladder	Annex III
WR Pointer, Augmented Overlay	Annex XI
Context-aware adaptive WR menus	Annex XII

Where this annex uses a term defined in one of those annexes, the definition there governs.

XIII.2 Inhomogeneity as an Architectural Condition

Corporate AI environments are inhomogeneous by construction. A single enterprise simultaneously operates devices of different attestation strength, networks of different exposure, data of different classification, workflows of different consequence, and models of different provenance, locality, and validation status.

The architectural consequence is not that model selection is difficult. It is that **the properties that determine what a component may do are not the properties that determine what it is good at.** A component may be the strongest available reasoner for a task and simultaneously ineligible to receive the task's context. A component may be narrow, weak, and slow in general terms and still be the only admissible provider for a step.

Conventional approaches resolve this by trusting an application layer to keep the two straight. This annex resolves it by making the two independent resolution stages (XIII.6) over independently governed properties (XIII.3), with the composition step performed by a component that holds no transmit authority at all (XIII.4).

This annex does not restate the Zero Trust argument. Locality-independent trust, and the prior work it stands on, are specified in Annex X.

XIII.3 Separation of Capability, Permission, Context Access, and Authority

Four properties are frequently conflated in conventional AI integrations. In this architecture they are independently governed and independently recorded.

Capability — the component is technically able to perform a function. **Permission** — the active governance instrument admits that function being invoked in this workflow. **Context access** — the component is admitted to receive a particular class of data. **Authority** — the component may cause an external effect.

The governing non-implications are normative:

```
Capability      ⇏  Permission
Permission      ⇏  Context Access
Context Access  ⇏  Execution Authority
Model quality   ⇏  any of the above
```

The fourth line is stated explicitly because it is the one most often violated in practice: the introduction of a more capable model *MUST NOT* widen its permission, context access, or authority. Capability upgrades and authority grants are separate governed events with separate records.

An adapter *MAY* specialize recognition. An adapter *MUST NOT* expand permission, context access, or authority. A capability request *MAY* ask for evidence. A capability request *MUST NOT* itself grant access to it (XIII.6.5).

XIII.4 The Composition Adapter

XIII.4.1 Definition

A **Composition Adapter** is an admitted local model adaptation — typically a LoRA adapter over a resident base model — whose declared capability is the construction of a typed **Evidence Object** from heterogeneous local signals.

Its function is explicitly *not* to answer the operator's question. It is the component that makes a question answerable elsewhere without exporting the environment that produced it.

Admitted input classes include:

- perception outputs: detections, class identifiers, confidences, tracks, masks, OCR regions;
- operator inputs: prompt text, selections, WR Pointer and Augmented Overlay interactions;
- machine signals: error logs, exception traces, status codes, deterministic sensor and PLC reads;
- workflow state: active WCR step, preconditions satisfied, prior approvals;
- retrieval results from the WR Graph™, within the requester's graph capability;
- device and session metadata.

XIII.4.2 The Completeness Invariant

Within the local trust domain, composition *MUST* be lossless with respect to admitted signals.

No signal admitted into composition may be discarded, summarized away, or merged in a manner that destroys per-field provenance or per-field classification. Where the adapter derives a statement from several signals, the derivation *MUST* be recorded as an additional field referencing its inputs; it *MUST NOT* replace them.

Three reasons make this invariant load-bearing rather than merely tidy:

1. **Later requests can only be satisfied from retained evidence.** The capability request loop (XIII.6.5) is only meaningful if the runtime still holds what a remote reasoner subsequently asks for. A lossy local summary silently converts a governed follow-up into a new capture, a new disclosure, and possibly a new consent event.
2. **Provenance can only be asserted for retained fields.** A summary that has absorbed its sources cannot carry their hashes, confidences, or capture times.
3. **Minimization is a policy decision, not a model output.** If the adapter decides what to drop, a probabilistic component has silently become the enforcement point for a governance boundary.

XIII.4.3 Separation of composition from minimization

The Composition Adapter *MUST NOT* perform minimization, redaction, masking, or recipient selection.

It emits into a local **Composition Pool** (compare the Session Composition Pool of Annex VIII), which is resident to the local trust domain. The Composition Adapter holds **no transmit authority**: it cannot address a recipient, cannot invoke an external service, and cannot cause a TRANSMIT finalizer.

Composition and projection are therefore performed by different components with different failure modes: a probabilistic component that must be complete, and a deterministic component that must be selective.

XIII.4.4 Evidence Object structure

Each field of an Evidence Object *MUST* carry at minimum:

Attribute	Content
`value`	the observation, in its declared type
`type`	schema-declared field type
`producer`	capability identifier of the producing provider
`producer_artifact`	model, adapter, or tool identifier plus artifact hash
`adapter_set`	the adapter residency state in effect at production time (XIII.5.4)
`confidence`	where the producer emits one; absent for deterministic producers
`captured_at`	capture timestamp
`classification`	labels assigned under the applicable WR Expert™ file
`retention`	retention class

Deterministic producers (sensor connectors, parsers, geometry engines, graph queries) *MUST* be distinguishable from probabilistic producers at field level. A conforming implementation *MUST NOT* present a model-derived value and a deterministic read in a form that erases this distinction.

XIII.4.5 Instruction-incapability of composed content

All text entering composition — OCR output, error-log content, operator free text, retrieved document text, and any free-text return from an external service — is **data**. It is never instruction.

Extraction *MUST* be instruction-incapable in the sense established in Annex IX: no content that has passed through composition may alter routing, projection, consent class, adapter residency, or finalizer selection. This closes the most direct injection path into an orchestrating system: a machine's own error log.

XIII.4.6 Recipient Projection

For each intended recipient, the deterministic policy layer computes a **Recipient Projection**: a subset of Evidence Object fields, each optionally transformed under the ten graded measures of Annex X (warn, block pending release, mask, pseudonymize, obfuscate, distribute, abstract, local-only, recipient-bound encryption, refuse).

Projection is deterministic, evaluated against the active WR Expert classification and the recipient's governance instrument — an Internal Handshake for an internal device or sandbox, a WR Handshake for a WR counterparty, an External Service Admission and its Invocation Assurance Class for a non-WR provider.

Normative properties:

- Every projection that crosses a trust boundary *MUST* emit a Boundary Event Record (Annex X). Off-device records carry digests, not content.
- Two recipients *MAY* receive disjoint projections of the same Evidence Object. No recipient is entitled to the object.

- Where Distribution is the selected measure, the split *MUST* be computed such that no single recipient receives a reconstructible whole; the join remains resident locally.
- A projection *MUST NOT* be widened by any party other than the policy layer, and never in response to a recipient's request within the same invocation (XIII.6.5).

The resulting discipline is stated as one principle:

> **Complete inward, selective outward.** Composition is lossless with respect to the local environment. Every projection out of it is minimal with respect to a single named recipient and a single declared purpose.

XIII.5 Adapter Admission, Routing, and Adaptation

This section specifies the lifecycle of the adapters themselves. It applies to Composition Adapters, to specialization adapters (customer-specific detection classes, domain vocabularies, layout authoring), and to the hot-swappable adapter set operated by WR Guard™.

XIII.5.1 Admission is a PoAC event

An adapter is a capability acquisition and therefore sits under PoAC™ (Annex III), at the top of the escalation ladder.

An adapter *MUST NOT* be introduced where a lower rung of the ladder — workflow compilation, prompt profile, RAG, validator, tool — demonstrably satisfies the observed gap. The evidence requirement applies: the gap *MUST* be backed by observable signals, not by an assertion that an adapter would be better.

XIII.5.2 Adapter Admission Record

Admission produces a signed **Adapter Admission Record** containing at minimum:

- adapter identifier and version;
- adapter artifact hash;
- base-model compatibility constraint: architecture, tensor shape expectations, tokenizer identity, admitted base-model hashes;
- declared capability: task class and output schema;
- declared class vocabulary, where the adapter narrows a label space;
- training provenance: dataset identifier and hash, annotation authority, date;
- **adaptation method declaration** (XIII.5.7.2);
- **training lineage reference**: the identifier of the Adaptation Record under which the artifact was produced (XIII.5.7.4);
- **residency constraint**: the lowest classification of any training material incorporated, which bounds admissible deployment (XIII.5.7.5);
- validation evidence: evaluation set hash and reported metrics;
- memorization evidence, where sanitization is claimed (XIII.5.7.5);
- publisher attestation level P0/P1/P2 (Annex VII);
- admitted trust domains and device classes;
- enforcement mode where the adapter serves a guard function (advisory or enforced);
- expiry and renewal conditions.

An adapter whose base-model compatibility constraint is not satisfied at load time *MUST NOT* be loaded. Compatibility is a verification, not an assumption.

An adapter presented without an adaptation method declaration and a training lineage reference *MUST NOT* be admitted in any environment that requires adaptation traceability. A dataset hash alone is not a sufficient account of what an artifact encodes (XIII.5.7.2).

XIII.5.3 Composition of adapters

Where two or more adapters are stacked, fused, or merged, the **combination** *MUST* hold its own Admission Record.

Merged adaptations can exhibit behavior validated for neither constituent, including label-space collisions and confidence miscalibration. Admission of the parts is therefore not admission of the whole. This is stated normatively because it is the most likely route by which an unvalidated capability enters an otherwise governed runtime.

XIII.5.4 Hot-swap and residency

Adapters *MAY* be swapped at runtime — WR Guard operates a hot-swappable set of which the Scam Watchdog™ is one member.

Constraints:

- A swap *MUST* be recorded as a runtime state change with time, initiating policy, and resulting residency set.
- A swap *MUST NOT* change the declared capability contract of an in-flight task. Where a task's declared contract can no longer be satisfied, the task *MUST* fail into a typed refusal rather than continue under a different capability.
- The residency set in effect at production time *MUST* be carried in the `adapter_set` attribute of every field produced during that residency (XIII.4.4). Provenance that names a base model but not the active adaptation is incomplete.

XIII.5.5 Routing among adapters

Adapter selection is capability resolution (XIII.6), not model preference. It is subject to the same eligibility gate and the same router discipline: selection occurs among pre-declared providers; the resolver never synthesizes a provider, a chain, or an adaptation at runtime (WCR principle, Annex IV/IX).

XIII.5.6 Adaptation triggers and Shadow Admission

Adaptation is orchestrated, not opportunistic.

Trigger evidence. Candidate signals include repeated low confidence over an identifiable class, unresolved or colliding class identifiers, validator rejections, operator corrections, and explicit operator feedback (thumbs plus description, as used by the Bayesian priority mechanisms of Annex VIII). These accumulate as PoAC gap evidence.

Proposal discipline. Accumulated evidence produces a *proposal* to adapt. Propose, never silently apply. No conforming implementation performs unattended adaptation of an admitted adapter.

Training data classification. Training material derived from an operator environment *MUST* inherit the classification of its sources under the applicable WR Expert file, and training *MUST* occur within a trust domain admitted for that classification. An adapter is a lossy but real encoding of its training data; adaptation is therefore a disclosure event and is governed as one.

Shadow Admission. A candidate adapter enters a shadow state in which it produces observations that are recorded with full provenance but *MUST NOT* contribute to any Recipient Projection, any operator-visible assertion, or any finalizer. Promotion out of shadow state requires a new Adapter Admission Record referencing the shadow evaluation evidence.

Idle-path constraint. Adaptation, evaluation, and optimization *SHOULD* execute on the idle path, not the real-time path, consistent with the optimization-loop discipline of Annex III and Annex VIII.

XIII.5.7 Adaptation as a Governed Execution

XIII.5.6 governs *whether* an adaptation is undertaken. This section governs *how it runs* and *what it may subsequently be*.

The distinction matters because a training run is not an operational side activity. It is the most context-hungry operation the architecture performs — it may read across entire WR Graph™ regions in a single pass — and it terminates in a portable artifact that is a lossy but real encoding of everything it read. An architecture that governs inference boundaries while leaving adaptation to an operations pipeline has left its widest channel unguarded.

XIII.5.7.1 Adaptation is an authorized execution

An adaptation run *MUST* be initiated as a governed execution under the finalizer lifecycle of Annex IX, not as an unattested background job.

- Initiation requires an explicit authorization: a signed superadmin policy, an operator consent event at the declared consent class, or both. The applicable consent class *MUST* be no lower than the class that would be required to disclose the training material to the trust domain in which the resulting artifact will reside.
- The training environment *MUST* be attested. Where hardware attestation is available it *MUST* be recorded; where it is not, the absence *MUST* be recorded rather than omitted (Honest Boundary, Annex X).
- The run's read access to the WR Graph, to file stores, and to operator archives is granted as a scoped capability like any other requester's, and is subject to the requester-class rule of Annex X. Training is a requester. It inherits no privilege from being internal.
- The run *MUST NOT* be able to widen its own scope mid-execution.

XIII.5.7.2 Method declaration

The following *MUST* be declared before the run and recorded with it:

- adaptation technique and library version;
- adapter rank, target modules, and any quantization applied to the base during adaptation;
- training objective and loss masking;
- optimizer, learning-rate schedule, batch composition, and step or epoch count;
- random seed and, where applicable, determinism settings;
- any regularization or privacy mechanism claimed (XIII.5.7.5);
- base-model artifact hash under which the run executed.

The rationale is specific rather than bureaucratic: two adapters produced from an identical dataset under different rank, objective, step count, or loss masking exhibit materially different memorization behavior. The dataset hash establishes what was available to the run. It does not establish what the artifact retained. Only the method declaration, taken together with the memorization evidence of XIII.5.7.5, supports any claim about the latter.

XIII.5.7.3 Reproducibility

Where an environment requires adaptation traceability, the run *SHOULD* be reproducible: re-execution under the recorded method, seed, base-model hash, and dataset *SHOULD* yield an artifact whose hash matches, or whose divergence is bounded and explained by declared non-determinism.

Where reproducibility cannot be achieved, that fact *MUST* be recorded as a property of the Adaptation Record rather than left implicit. An unreproducible run may still be admitted; it may not be presented as though it were verifiable.

XIII.5.7.4 The Adaptation Record

Each run produces a signed **Adaptation Record** containing at minimum: the authorizing policy or consent event; the initiating identity; the attestation state of the training environment; the method declaration of XIII.5.7.2; the set of source identifiers admitted into the run, with their classifications; the sample-level lineage reference or, where sample-level lineage is not maintained, an explicit declaration of that limitation; start and completion times; the resulting artifact hash; and the resulting residency constraint (XIII.5.7.5).

The Adaptation Record is verifiable at tiers L, C, and X on the same terms as other execution evidence (Annex IX). Off-device records carry digests, not training content.

Every subsequent Adapter Admission Record *MUST* reference the Adaptation Record that produced its artifact. An artifact whose lineage terminates in an unrecorded run is an artifact of unknown provenance regardless of how well it performs.

XIII.5.7.5 The residency constraint

This is the constraint that closes the channel.

> An adapter *MUST NOT* reside in, or be transferred to, a trust domain in which the lowest-classified element of its training material could not lawfully have been read at the time of that training.

Without this rule, adaptation is a laundering path: classified content is read in a high domain, encoded into weights, and the weights are then deployed to a field device or an external partner as a mere capability update. No Boundary Event Record fires, because at the moment of transfer only parameters moved. The disclosure occurred earlier, in a form the boundary machinery does not inspect.

Consequences:

- The residency constraint is computed at admission from the Adaptation Record and carried in the Adapter Admission Record. It is a property of the artifact, not of the deployment request.
- Transfer of an adapter across a trust boundary is a disclosure event and *MUST* emit a Boundary Event Record naming the residency constraint, exactly as a data transfer would.

- Merged adapters (XIII.5.3) inherit the *most restrictive* residency constraint of their constituents. A merge cannot dilute classification.
- A constraint *MAY* be relaxed only against **memorization evidence**: a declared sanitization mechanism — differentially private training, canary-based extraction testing, targeted memorization audit, or an equivalent — whose method and results are recorded in the Adaptation Record and referenced from the Admission Record. The relaxation is then admitted on the strength of that evidence and is revocable if the evidence is later invalidated.
- Sanitization *MUST NOT* be assumed from the mere fact that adaptation is lossy. Loss is not sanitization.

XIII.5.7.6 Operator-derived trigger material

The trigger signals of XIII.5.6 — operator corrections, free-text feedback, rejected outputs — are themselves operator-derived content and carry classification. They *MUST NOT* enter a training set on the assumption that feedback is metadata. Where such material is admitted into a run, it appears in the Adaptation Record's source set with its classification like any other source, and it participates in the computation of the residency constraint.

XIII.6 Capability Resolution

XIII.6.1 Provider declaration

Every provider that may participate in resolution *MUST* hold a current **Provider Declaration**. A provider without one is ineligible by definition; absence of a declaration is not a neutral state, and *MUST NOT* be treated as an unknown to be resolved at runtime.

Provider classes are not limited to language models. They include:

- resident base models and admitted adapters;
- specialist perception, tracking, segmentation, OCR, embedding, and reranking models;
- **spatial and world models** that maintain a persistent scene, environment, or dynamics representation;
- **generation pipelines** exposed as a single endpoint while internally composed of retrieval, reranking, generation, filtering, and post-processing stages;
- external reasoning providers;
- deterministic tools, connectors, solvers, and validators;
- data sources, including WR Graph™ regions;
- human approval roles.

A Provider Declaration contains at minimum: provider identifier and version; the governing instrument (XIII.4.6); declared capability and output schema; the projection fields the provider is declared to accept; statefulness class (XIII.6.1.1); retention declaration; sub-provider disclosure (XIII.6.1.3); and the evidence tier obtainable from the provider.

XIII.6.1.1 Statefulness classes

Class	Property
S0	Stateless. No retention across invocations.
S1	Session-stateful. State bounded by a declared session and discarded at its end.

Class	Property
S2	Persistent-stateful. Accumulates representation across sessions — environment maps, provider-side memory, conversation history, or adaptation on submitted content.

For S1 and S2 providers, per-invocation minimization does not bound total disclosure. The policy layer *MUST* evaluate a Recipient Projection against the **cumulative** disclosure already made to that provider, not against the current invocation alone. An S2 provider *MUST NOT* receive a projection whose accumulation exceeds what the operator could have disclosed in a single act at the attained consent class.

Where a provider's statefulness cannot be verified, it *MUST* be treated as S2.

XIII.6.1.2 Spatial and world models

A spatial or world model that receives a sequence of captures accumulates a reconstruction of the physical environment. The resulting representation may be substantially more sensitive than any single capture that produced it: a facility layout is disclosed by no individual frame and by all of them together.

Such providers are therefore **S2 by default**. A conforming deployment *SHOULD* prefer a resident spatial model. Where an external spatial provider is admitted, projections *SHOULD* be computed over derived geometry, poses, and relations rather than raw capture wherever the task permits, and the cumulative rule of XIII.6.1.1 applies over the session sequence rather than the individual frame.

XIII.6.1.3 Composite providers

A provider that internally invokes retrieval, further models, or third-party services *MUST* either enumerate its stages and their sub-providers in its declaration, or declare itself **opaque**.

An opaque composite provider is treated as S2 and as having undisclosed sub-recipients; its eligibility is bounded accordingly, and projections to it are evaluated as disclosures to an unnamed set. Opacity is thereby a declared and priced property rather than an unexamined assumption.

XIII.6.2 Eligibility precedes suitability

Let a task `T` require capabilities `C_T = {c_1 … c_n}`, and let `P` be the set of declared providers (XIII.6.1).

For each required capability, the runtime first evaluates a hard predicate over the active context `X_T`:

```
Eligible(p, c, X_T) ∈ {0, 1}
```

`X_T` includes at minimum: the presence of a current Provider Declaration, data locality constraints, classification of the required context, the governing instrument for the provider, the consent class attained, attestation level, statefulness class together with cumulative disclosure already made to that provider, and the availability of the verification evidence the workflow requires.

Only providers for which `Eligible = 1` enter selection. Providers for which `Eligible = 0` are not ranked, not scored, and not offered as fallbacks.

This annex deliberately does not constrain the selection function applied to the eligible set. Weighted scoring over quality, latency, cost, and specialization is well-established prior work (XIII.8) and nothing here depends on a particular formulation. What is specified is the ordering:

eligibility is a gate, not a weight. A provider's permission to participate is never traded off against its quality.

XIII.6.3 Router discipline

The resolver selects among pre-declared providers and pre-declared chains. It binds parameters and chooses branches within declared constraints. It *MUST NOT* create a path that was not declared. The AI participates in orchestration as a router over a fixed topology, never as an author of topology.

XIII.6.4 Routing proposals

The router discipline of XIII.6.3 constrains execution, not authorship. The orchestrator *MAY* propose routings — including best-practice compositions derived from recorded outcomes of prior resolutions, quality, latency and cost evidence, operator feedback, and idle-path optimization loops (Annex III, Annex VIII).

A proposal is not a path.

- A proposed routing *MAY* be presented, recorded, and evaluated in shadow. It *MUST NOT* execute before admission as a declared chain.
- A proposal *MUST* carry its supporting evidence, so that admission is a review of grounds rather than of a recommendation.
- Propose, never silently apply.
- A signed superadmin policy *MAY* pre-authorize recomposition within an already-admitted provider set, provided the recomposition introduces no new recipient, no new provider, and no widening of any Recipient Projection. Recompositions admitted under such a pre-authorization are recorded individually.

The topology therefore grows by governed admission rather than by runtime synthesis. The orchestrator is permitted to become better at routing over time without ever becoming the authority that decides what routes exist.

XIII.6.4.1 Proposal authority and the bounded proposal space

The freedom to propose is itself governed. Two constraints bound it.

Proposal generation is a PoAC™ act. A routing proposal is a capability-acquisition proposal in the sense of Annex III and inherits that discipline: it *MUST* rest on observable gap evidence rather than on an assertion that an alternative would be better, it *MUST* respect the escalation ladder — a proposal to engage a further provider is not admissible where a lower rung demonstrably closes the observed gap — and it is subject to propose-never-silently-apply throughout.

The proposal space is bounded by the active policy. The orchestrator *MUST NOT* propose a routing that could not be admitted under the policy in force for this operator, workflow, and data classification. Proposals are generated within the admissible set, not filtered after generation.

This second constraint is not merely economical. A proposal naming a provider that the operator may not lawfully engage discloses the existence and attractiveness of an inadmissible path, and applies pressure toward obtaining an exception. An orchestrator that surfaces what policy forbids has become an advocate for widening its own scope.

Admission authority is declared, not assumed. The role competent to admit a proposed routing is fixed by policy for each proposal class. An operator's authority to *use* an admitted routing does not imply authority to *admit* a new one. Where the two differ, the proposal is presented to the operator as information and routed to the competent role for admission.

XIII.6.4.2 Presentation of proposals

A proposal that is not legible to the party deciding on it is not a proposal. Where a routing proposal is surfaced to an operator or an approval role, it *MUST* state:

- the providers it would engage, and the governing instrument applying to each;
- the Recipient Projections that would result, identifying every field that would be disclosed under the proposal but is not disclosed under the routing currently in force;
- the trust boundaries that would be crossed, and the statefulness class of each recipient;
- the delta against the current routing, rather than the proposed routing in isolation;
- the role competent to admit it, where that role is not the party being shown the proposal (XIII.6.4.1).

A proposal *MUST* state its **disclosure consequence** alongside its grounds. Recorded quality, latency, and cost evidence is not a sufficient account of a routing: a recomposition that improves latency by widening egress *MUST* be presented as doing both, and an optimization loop *MUST NOT* select against quality, latency, or cost alone.

Proposals are surfaced through the admitted rendering path — declared resources, typed slots, declared links, no runtime-synthesized content (Annex VIII, Annex XI). A proposal is content like any other and receives no rendering privilege.

A proposal *MUST NOT* be pre-selected, presented as already approved, or arranged such that inaction results in adoption. Offer, never action.

Rejected proposals are recorded together with their rejection. Re-proposal of a rejected routing without new supporting evidence is non-conforming.

XIII.6.4.3 Alternative paths

A single recommendation invites acceptance. A comparison invites a decision. Where more than one admissible routing exists for a task, the orchestrator *SHOULD* present them as a set rather than surface one.

A comparative presentation *MUST*:

- include the routing currently in force, or explicit continuation without change, as a named option — the status quo is an alternative and is presented as one;
- state, for each alternative, both its advantages and its disadvantages on the same dimensions, so that no option is described only by what commends it;
- report each alternative's disclosure consequence alongside its quality, latency, and cost grounds (XIII.6.4.2), and identify which alternative discloses least;
- retain an alternative that is weaker on quality, latency, or cost but materially lower in disclosure. Such an option *MUST NOT* be omitted, collapsed into another, or presented as dominated;
- name the trade-off explicitly where dimensions conflict, rather than resolving it silently through ordering or emphasis.

A comparative presentation *MUST NOT* rank the set by a single composite score, and *MUST NOT* mark one alternative as recommended in a form that carries default acceptance (XIII.6.4.2). Ordering is a presentational choice and *MUST NOT* substitute for the statement of trade-offs.

Where only one admissible routing exists, that fact *SHOULD* be stated as such. "No alternative is admissible under the active policy" is itself information the deciding party is entitled to, and is distinguishable from "no alternative was examined".

XIII.6.5 The Capability Request loop

An external reasoning provider *MAY* return a **Capability Request** — a request for a further image region, a second viewing angle, OCR of a specific label, a fresh sensor read, a document section, or a validation of an identifier.

Normative properties:

- A Capability Request is **data**. It carries no authority whatsoever.
- It is resolved by the local runtime against: whether the capability exists, whether it is declared for this workflow, whether the required context may be projected to that recipient, and whether the attained consent class covers the resulting disclosure.
- A Capability Request *MUST NOT* raise a consent class, widen a Recipient Projection, or cause an adapter swap.
- An unsatisfiable request returns a **typed refusal** to the requester. Silent non-fulfilment is non-conforming, because it makes governance indistinguishable from failure.
- Each satisfied request produces its own Boundary Event Record.

The loop is therefore:

```
local capabilities → E₁ → external reasoning → capability request
                   → runtime resolution → E₂ → refined reasoning
                   → proposal → approval → governed finalizer
```

Refinement is achieved by iterated minimal disclosure rather than by granting an agent access to the environment.

XIII.6.6 Authority remains outside the composition

External effects occur only through governed finalizers — READ, TRANSMIT, CHANGE, PERSIST, EXECUTE — under the lifecycle and consent classes of Annex IX, with verification at tiers L, C, and X. No component described in this annex holds execution authority: the Composition Adapter cannot transmit, the specialist providers cannot address the environment, the external reasoner cannot act, and the resolver selects but does not execute.

XIII.7 Worked Example

A technician directs a device camera at an unfamiliar pump while the machine controller is emitting error codes.

Step	Provider	Governing reason
Detect components	local vision model	latency; no continuous image egress

Step	Provider	Governing reason
Recognize customer-specific pump class	admitted specialization adapter	narrow validated class vocabulary, stable metadata
Maintain object identity	specialist tracker	temporal consistency
Read warning label	specialist OCR	exact extraction with region coordinates
Read controller error log	deterministic connector	authoritative, non-probabilistic
Read inlet pressure	deterministic PLC connector	authoritative machine data
Compose evidence	**Composition Adapter**	complete inward fusion with per-field provenance
Project to external reasoner	deterministic policy layer	minimization, classification, Boundary Event Record
Compare failure hypotheses	external reasoning provider	broad knowledge; governed by External Service Admission
Validate proposed threshold	deterministic validator	exact numerical and policy validation
Approve	operator, at required consent class	human authority remains explicit
Execute	governed finalizer under WR TED	verifiable external effect

The Evidence Object retains the full error-log text, the full detection set, the OCR region and its coordinates, the pressure read and its PLC tag, and the operator's own phrasing — each with producer, artifact hash, active adapter set, confidence, and classification.

The projection transmitted to the external reasoner contains the machine class, the normalized error code, the abstracted pressure state, and the OCR string. It does not contain the operator identity, the site identifier, the raw frame, the PLC tag path, or the unrelated log lines that shared the buffer.

The reasoner proposes cavitation, inlet obstruction, or sensor fault, and requests a fresh inlet reading plus a crop of the valve position. The request is resolved locally: the sensor read is declared and is satisfied deterministically; the crop is a new disclosure of imagery and is either shown to the operator for release or transmitted automatically only where the admission and consent class already cover it. Both outcomes are recorded.

Nothing in the loop granted the reasoner access to anything. It received two answers to two questions it was permitted to ask.

XIII.8 Prior Work and Disclosure Position

This annex makes **no novelty claim** over its constituent mechanisms. It is published as a defensive disclosure.

Substantial prior and concurrent work exists in each contributing area, including: zero-trust architecture and locality-independent authorization; data loss prevention, CASB, and information-flow control including DIFC and mandatory access control; mixture-of-experts and learned routing; parameter-efficient adaptation and adapter routing, composition, and serving (including work in the AdapterFusion, LoraHub, PHATGOOSE, and S-LoRA lines); cost- and quality-aware model routers (including the FrugalGPT and RouteLLM lines); tool-using and agentic frameworks with capability

declaration; sensor and multi-modal fusion with provenance metadata; and supply-chain attestation for model artifacts.

What is disclosed here is a **specific combination**, and the disclosure is intended to cover that combination as a whole:

1. an admitted local adaptation whose declared capability is evidence construction rather than answering, holding no transmit authority;
2. a completeness invariant inward paired with deterministic per-recipient minimization outward, with the two performed by different components;
3. per-field provenance that names the active adapter residency, not merely the base model;
4. mandatory declaration of every participating provider, with statefulness classes that make cumulative disclosure — not per-invocation minimization — the governing quantity for stateful providers, and declared opacity for composite pipelines;
5. eligibility evaluated as a hard gate strictly prior to any quality-based provider selection, with an orchestrator permitted to propose better routings within a policy-bounded proposal space under capability-acquisition discipline — surfaced as a comparative set including the status quo, each alternative carrying its disclosure consequence and its disadvantages, never ranked by a single composite score, never pre-selected, never executable before admission — but never to author executable topology;
6. non-authorizing capability requests from external reasoners, resolved locally, with typed refusal;
7. adapter admission, merge admission, hot-swap recording, shadow admission, and evidence-triggered adaptation proposals bound to a capability-acquisition discipline that treats training data as classified disclosure;
8. adaptation executed as an authorized, attested, scope-bounded run with a mandatory method declaration and a signed Adaptation Record referenced by every downstream admission;
9. a residency constraint derived from training-material classification that binds where the resulting artifact may reside, treats adapter transfer as a disclosure event, propagates most-restrictively through merges, and is relaxable only against recorded memorization evidence;
10. the whole operating under cryptographically verifiable handshake, internal handshake, and external service admission instruments, with boundary records and finalizer-level effect control.

No claim is made that any single element of this list is unprecedented.

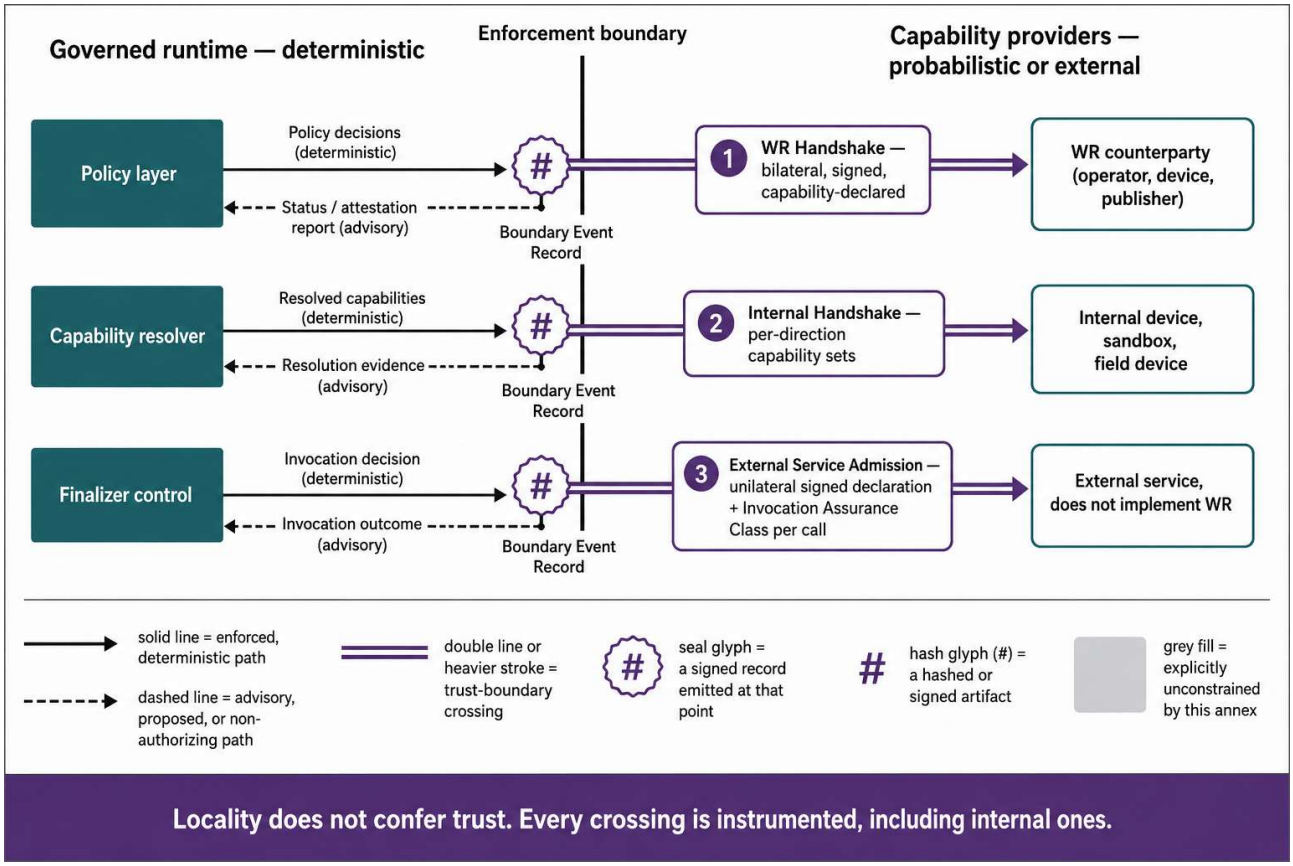
XIII.9 Open Items

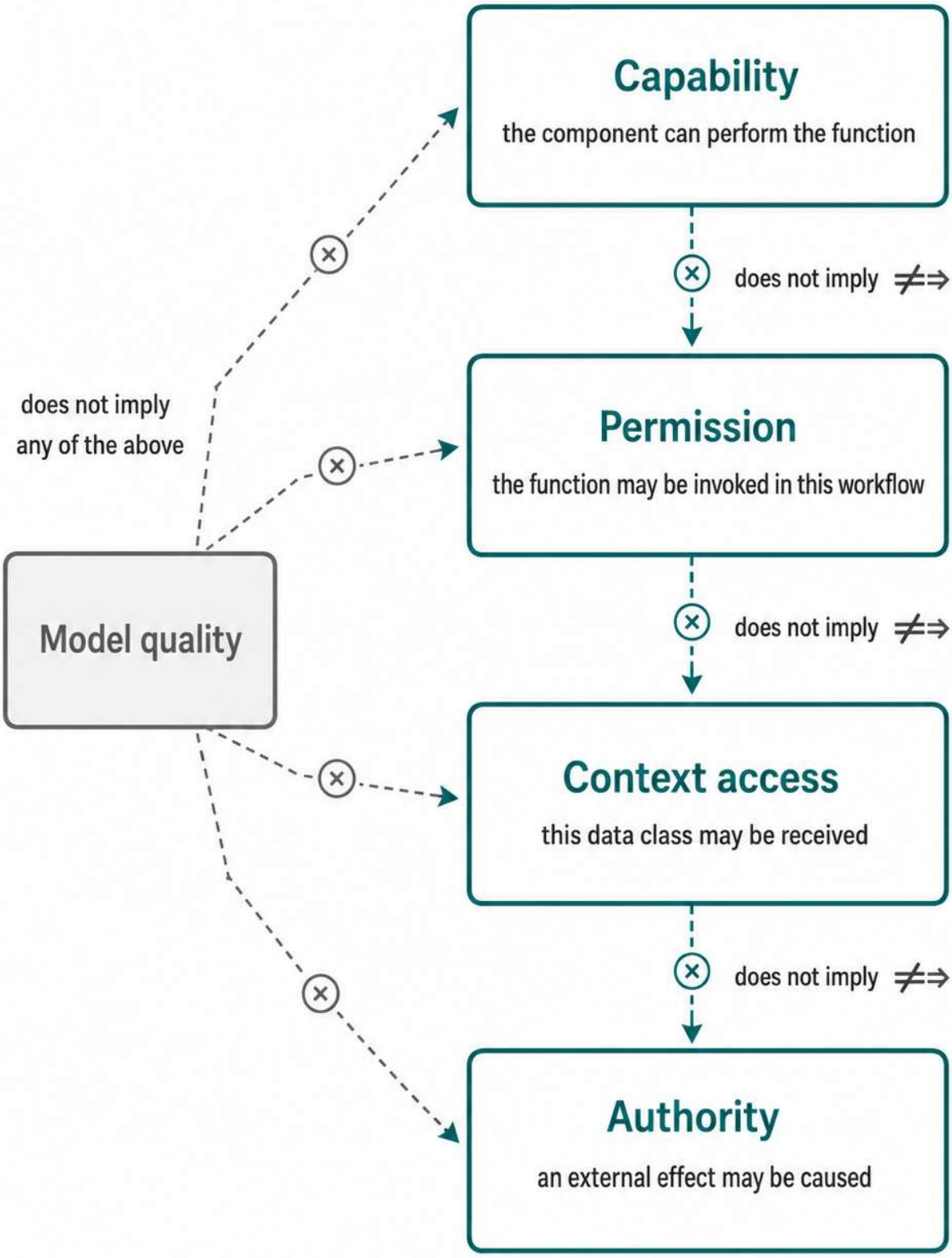
1. **Designation.** "Composition Adapter" is descriptive and provisional. A registered designation in the WR series has not been fixed.
2. **Structural placement.** This annex is closely adjacent to Annex X. It is published standalone because the adapter lifecycle is a distinct subject matter from the egress guard, and because separate documents date and reference more cleanly. Merging remains possible.
3. **Evidence Object schema.** The field attributes of XIII.4.4 are a minimum set, not a wire format. A canonical schema and its versioning rules are not yet fixed.
4. **Merge validation methodology.** XIII.5.3 requires admission of adapter combinations but does not prescribe how a combination is validated.

5. **Projection verifiability.** Whether a recipient can be given cryptographic evidence that its projection was computed under a declared policy, without disclosing the withheld fields, is open.
6. **Memorization evidence methodology.** XIII.5.7.5 admits relaxation of a residency constraint against recorded sanitization evidence but does not prescribe an evaluation methodology or an acceptance threshold. Prescribing one prematurely would be worse than leaving it declared and reviewable.
7. **Sample-level lineage.** XIII.5.7.4 permits an explicit declaration that sample-level lineage is not maintained. The cost and feasibility of maintaining it for large operator-derived corpora, and whether a Merkle construction over the training set is the appropriate mechanism, are unresolved.

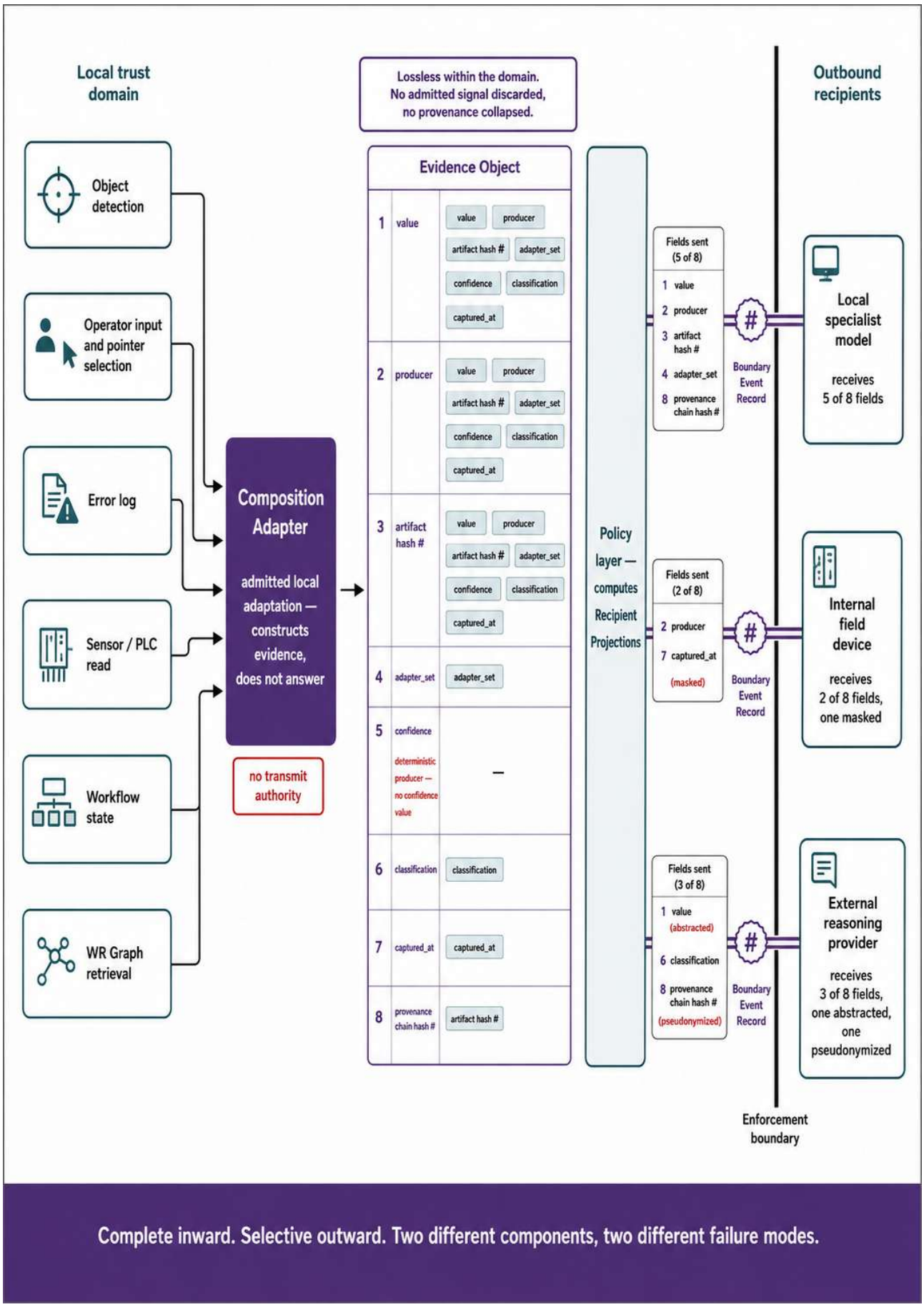
XIII.10 Version History

v1.0 — 25 July 2026. Initial publication.



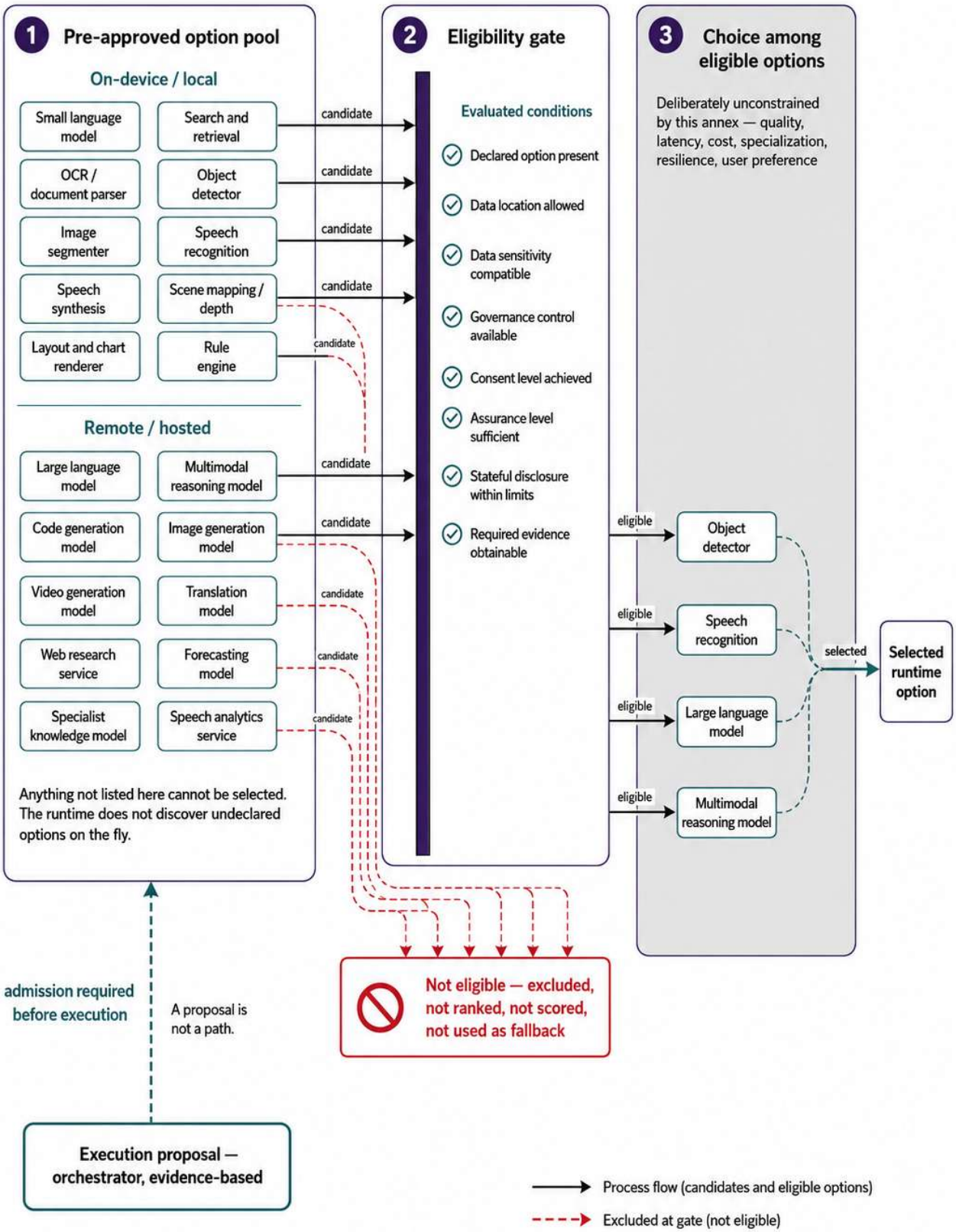


Each is granted by a separate governed event and carries its own record.



Complete inward. Selective outward. Two different components, two different failure modes.

Eligibility is a gate, not a weight.



RANKED ORCHESTRATION FOR ONE ISSUE

The orchestrator ranks admissible routings and suggests the best profile. Alternatives remain visible.

Example issue

Input: machine photo + short spoken note

Task: diagnose a pump / control-panel fault

Mode: suggest best + show alternatives

Auto mode: optional — run rank 1 if policy allows

Ranking profile for this issue

Capability fit	35%
Disclosure / egress	25%
Latency	20%
Cost	10%
Local-preference policy	10%

Excluded before ranking

Not allow-listed under active WR Handshake permissions or local policy

Local ingest → Unapproved analytics provider → Overseas vision service → External recipient

Not proposed. Not surfaced.

Operator view (admissible set)

For the same issue, more than one admissible solution may exist.
Candidates are generated only from allow-listed routings.

Rank reflects issue-specific fit.

Orchestrator ⇢ generate → score → rank → suggest →

1. Local OCR + local vision specialist + resident planner — Suggested

Local OCR → Local vision model → Resident planner → Operator reply

Capability high | Disclosure low | Latency good | Cost good

Total score: 8.8 / 10

Why here: Best balance for this photo-based fault on a known machine family.

Admitted by: WR Handshake + local policy

2. Local OCR + cloud multimodal model — Alternative

Local OCR → Cloud multimodal AI → Operator reply

Capability fit very high | Disclosure medium | Latency medium | Cost medium

Total score: 8.1 / 10

Why here: Stronger open-world analysis, but requires image and text egress.

Admitted by: WR Handshake + local policy

3. Local speech-to-text + local OCR + cloud multimodal + retrieval — Alternative

Local speech-to-text → Local OCR → Cloud multimodal AI → Allowed knowledge retrieval → Operator reply

Capability breadth high | Disclosure higher | Latency slower | Cost higher

Total score: 7.5 / 10

Why here: Useful when the voice note and equipment history matter.

Admitted by: WR Handshake + local policy

4. Fully local speech + OCR + vision + resident planner

Local speech-to-text → Local OCR → Local vision model → Resident planner → Operator reply

Disclosure least | Capability narrower | Latency medium | Cost medium-high

Total score: 7.1 / 10

Why here: Kept transparent because privacy is strongest, even if fit is weaker here.

Admitted by: local policy

Retained as a valid alternative. Lower disclosure can outweigh weaker performance in some contexts.

Operator choice mode

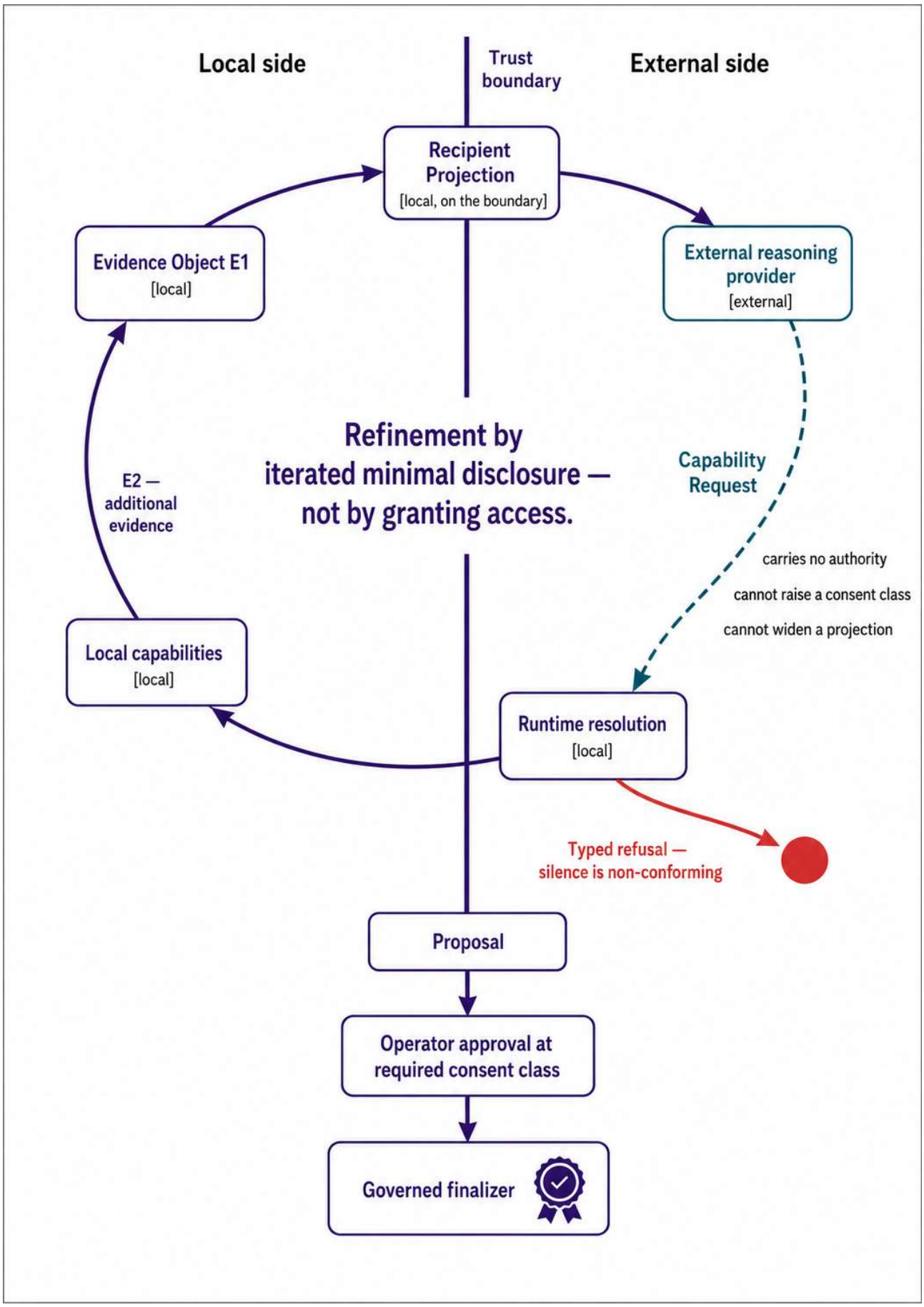
Show rank 1 first, but keep ranked alternatives transparent.

Automatic mode

If settings and policy permit, the orchestrator executes the highest-ranked admissible routing automatically.

Admission is not selection. Allow-listed routings may still rank differently per issue.

If only one routing is admissible, state that explicitly. This differs from saying no alternatives were examined.


Local side
Trust boundary
External side
Recipient Projection
[local, on the boundary]
Evidence Object E1
[local]
External reasoning provider
[external]
Refinement by iterated minimal disclosure — not by granting access.
E2 — additional evidence
Capability Request
carries no authority
cannot raise a consent class
cannot widen a projection
Local capabilities
[local]
Runtime resolution
[local]
Typed refusal — silence is non-conforming
Proposal
Operator approval at required consent class
Governed finalizer

EXAMPLE: INCIDENT TICKET DRAFT RESPONSE

Goal: Produce a draft response while minimizing disclosure, latency and cost.

ALLOWED SET — DETERMINISTICLY DEFINED

Routings are generated only from capabilities allow-listed by the active WR Handshake permissions and local policy. Nothing outside this set is considered.

BEST SUGGESTION (by current policy weights)
Option C — Local composition + external reasoning provider
Score: 0.78 / 1.00

Routing alternatives for the SAME ISSUE — multiple valid solutions may apply.

A — Current routing — continue without change

status quo

Local Classifier → Local RAG → Local Generator

Advantages	Disadvantages
• Lowest latency • Lowest cost • No external egress • Full residency	• Lower quality on complex issues • Class collisions may persist

Disclosure profile (fields)

CID	UID	TKT	PRO	LOG	ATT	KB	MET
■	□	□	□	□	□	□	□

+2 fields vs current baseline

Quality (est.)	★★★☆☆
Latency	Low
Cost	Low
Disclosure	Low
Score (weighted)	0.46 / 1.00

B — Local specialist + resident reasoning

discloses least (measured)

Local Classifier → Local Specialist → Local Reasoner

Advantages	Disadvantages
• Minimal disclosure • No external egress • Domain expertise grounded locally • Full residency	• Slower on complex reasoning • Higher local compute cost

Disclosure profile (fields)

CID	UID	TKT	PRO	LOG	ATT	KB	MET
■	■	□	□	□	□	□	□

−3 fields vs current baseline

Retained. Must not be omitted, collapsed, or presented as dominated.

Quality (est.)	★★☆☆☆
Latency	Medium
Cost	High (local)
Disclosure	Very Low
Score (weighted)	0.62 / 1.00

C — Local composition + external reasoning provider

RECOMMENDED

Local Classifier → Local Tools & Retrieval → Cloud Reasoner

Advantages	Disadvantages
• Highest quality on complex issues • Balanced latency • Balanced disclosure • Limits egress to curated summary	• Some external egress of projected data • Depends on provider availability

Disclosure profile (fields)

CID	UID	TKT	PRO	LOG	ATT	KB	MET
■	■	■	■	■	■	■	□

+1 field vs current baseline

Quality (est.)	★★★★☆
Latency	Medium
Cost	Medium
Disclosure	Low
Score (weighted)	0.78 / 1.00

D — Extended external pipeline

Cloud Pipeline → Cloud Specialist → Cloud Generator

Advantages	Disadvantages
• Potentially highest quality • Elastic at scale	• Highest latency • Highest cost • Widest egress • S2 recipient exposed

Disclosure profile (fields)

CID	UID	TKT	PRO	LOG	ATT	KB	MET
■	■	■	■	■	■	■	■

+5 fields vs current baseline

Quality (est.)	★★★★☆
Latency	High
Cost	High
Disclosure	High
Score (weighted)	0.32 / 1.00

Scoring (current policy weights)
Quality 40% Latency 20% Cost 20% Disclosure 20%

■ Best suggestion (by score) □ Measurement (not endorsement) □ Not selected

Operator may choose a different option. Orchestrator may auto-select if policy allows.

Proposals are generated within the allowed set (by WR Handshake permissions and local policy). not filtered after generation.

INADMISSIBLE ROUTING (OUTSIDE ALLOWED SET)

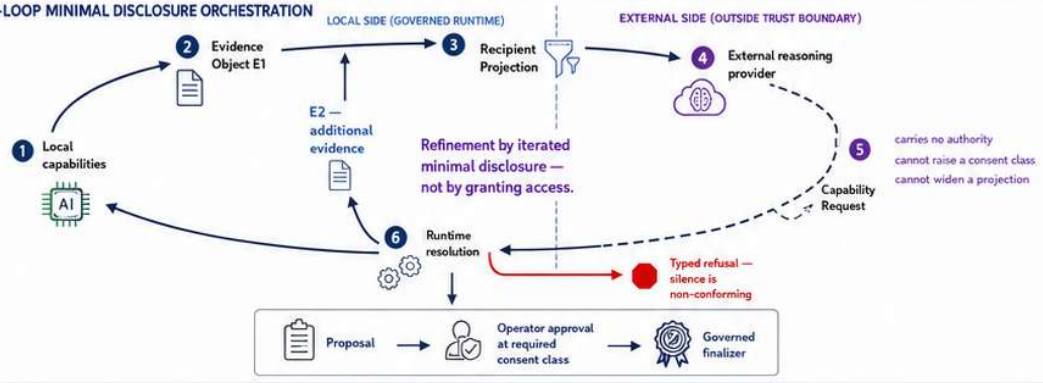
Inadmissible under active policy — not proposed, not surfaced.

Public Model → Unvetted Tool → Third-Party Storage

Reasons (examples):

- Not allow-listed by WR Handshake
- Local policy denies egress
- Consent class cannot be met
- Residency constraint violated
- S2 recipient not permitted
- Capability not declared

CLOSED-LOOP MINIMAL DISCLOSURE ORCHESTRATION



LIFECYCLE: GOVERNED ADAPTER ORCHESTRATION (ADAPTATION IS AN IDLE-PATH ACTIVITY)

REAL-TIME PATH (NO ADAPTER ACTIVITIES)

1 Trigger evidence
- repeated low confidence
- colliding class identifiers
- validator rejections
- operator corrections

Accumulates as PoAC gap evidence

2 Escalation check

An adapter must not be introduced where prompt profile, retrieval, validator or tool demonstrably closes the gap.

3 Proposal

Propose, never silently apply.

4 Authorized adaptation run
- consent class ≥ disclosure class of the training material
- attested training environment
- scoped read capability — training is a requester
- rank · target modules · objective · loss masking · optimizer · steps · seed · base hash

5 Adaptation Record

source set with classifications · lineage · reproducibility status · resulting artifact hash #

6 Adapter Admission Record

references the Adaptation Record · base-model compatibility verified at load · residency constraint carried

7 Shadow state → Residency

SHADOW
observations recorded with full provenance — contribute to no projection, no operator-visible assertion, no finalizer.
DEAD END

GATE
promotion requires a new Admission Record referencing shadow evidence.

RESIDENCY

IDLE PATH (ADAPTER GOVERNANCE PATH)

Where only one routing is admissible, that fact is stated — distinguishable from "no alternative was examined".

DISCLOSURE FIELDS (examples)

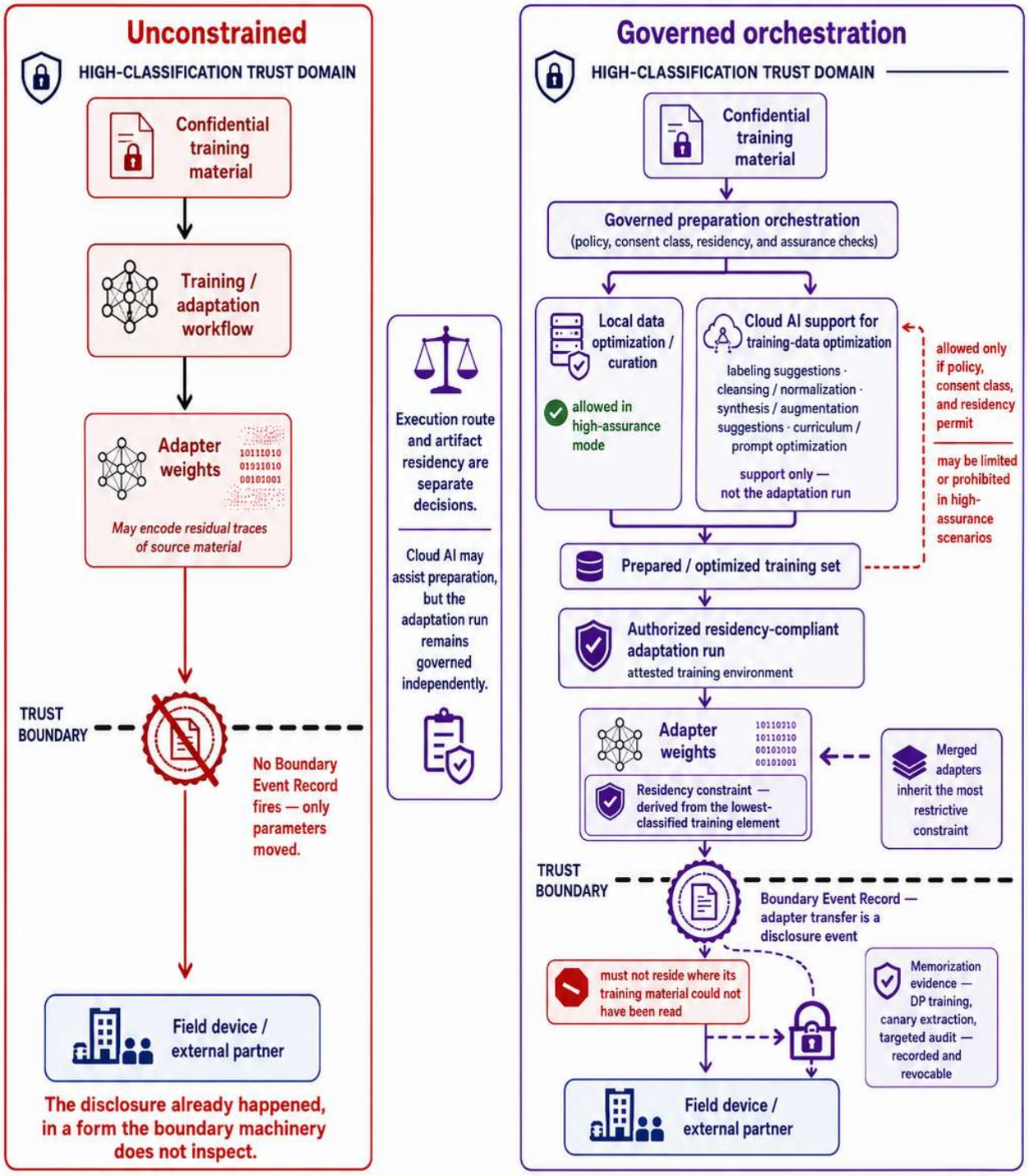
CID	UID	TKT	PRO	LOG	ATT	KB	MET
Customer ID	User ID	Ticket ID	Product Context	System Logs	Attachments / Metrics	Knowledge Bases	Derived Metadata

DISCLOSURE INDICATOR
■ disclosed
□ not disclosed

CONSENT CLASSES
S0 (none) S1 S2 S3

Governed finetuning orchestration under residency constraints

Cloud AI may assist training-data optimization and preparation, while the adaptation run itself remains governed by residency and assurance constraints.
Training data and resulting weights are distinct disclosure events.

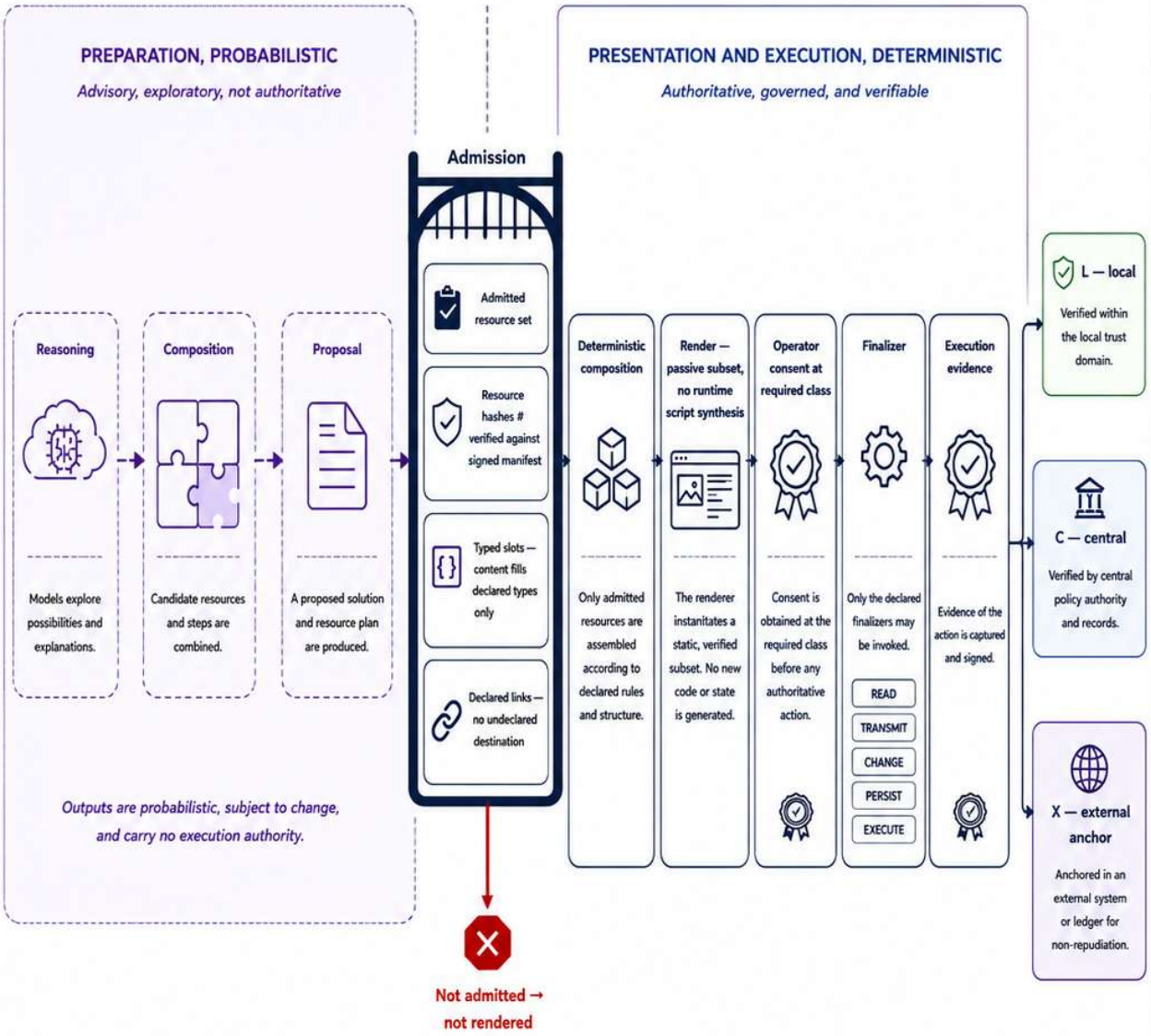


Loss is not sanitization.

Cloud assistance in data preparation does not remove disclosure obligations, and adapter weights remain governed artifacts.

GOVERNED RUNTIME PIPELINE — ADVISORY TO AUTHORITATIVE

From probabilistic preparation to deterministic execution under declared policy and consent.



The boundary is not remembered by the model.
It is enforced outside it.

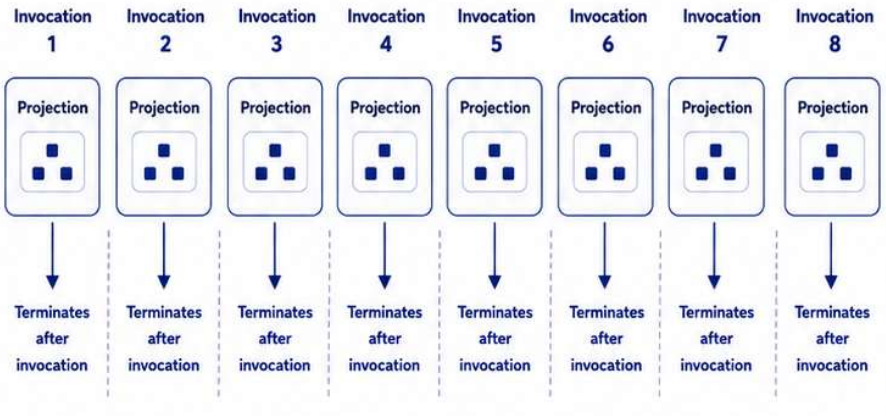
CUMULATIVE DISCLOSURE OVER SUCCESSIVE INVOCATIONS

Eight successive invocations of the same projection to the same provider

S0 stateless provider

No retention across invocations.

Each projection stands alone and terminates.

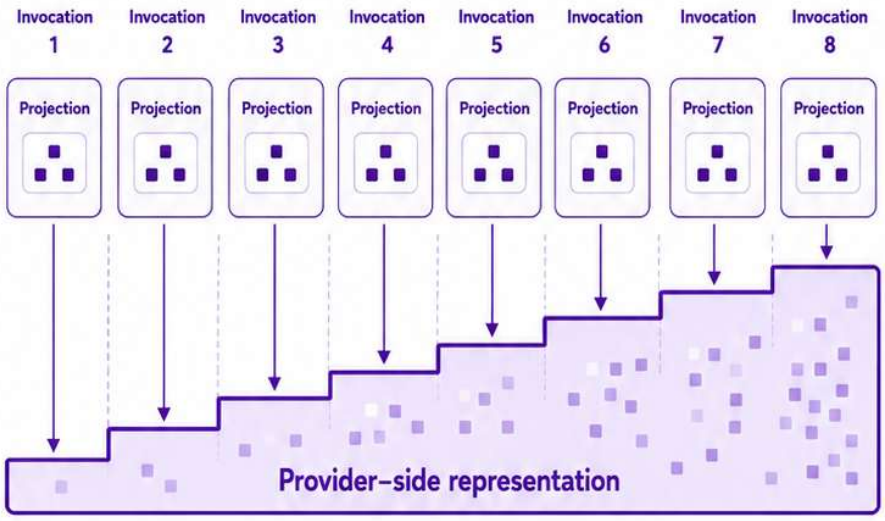


TOTAL DISCLOSURE = the largest single projection.

S2 persistent-stateful provider

Retains and integrates information across invocations.

Provider-side representation accumulates.



TOTAL DISCLOSURE = the accumulation. A facility layout is disclosed by no single frame and by all of them together.

Projections are evaluated against cumulative disclosure to that provider, not against the current invocation.

Composite provider declaring itself opaque is treated as S2 with undisclosed sub-recipients.

- **S0** — no retention across invocations
- **S1** — state bounded by a declared session
- **S2** — accumulates across sessions

Unverifiable statefulness is treated as S2.

In high-assurance environments, the security model cannot depend solely on trusting external providers when they claim that data is not stored, retained, or reused. Instead, the system should follow a trustless verification principle in which relevant facts, policy decisions, data transfers, and execution steps are cryptographically provable, auditable, and independently verifiable. Trust may remain useful operationally, but in critical environments it is not a sufficient control by itself.